☁️ CloudNotes

Product Requirements Document

v2.0 · Março 2026

SaaS · Supabase · Google OAuth · Docker Swarm · PWA

---

1. Visão Geral do Produto

CloudNotes é um aplicativo SaaS de notas minimalista, multi-usuário e auto-hospedado, inspirado no Simplenote. O objetivo é oferecer uma experiência de escrita rápida, limpa e confiável, com sincronização em tempo real entre dispositivos, instalável como PWA em qualquer plataforma.

O sistema é composto por três camadas principais:

- API REST (Node.js + Fastify) — lógica de negócio, planos, cotas e orquestração do Supabase
- Frontend Web / PWA (React + Vite) — interface responsiva, instalável como app no mobile e desktop
- Supabase — PostgreSQL gerenciado, Auth (Google OAuth), Storage (anexos) e Realtime (sync entre abas)

> ⚡ **Mudanças em relação à v1.0**
> • Banco: PostgreSQL self-hosted → Supabase (gerenciado)
> • Auth: JWT próprio → Supabase Auth com Google OAuth
> • Storage: MinIO self-hosted → Supabase Storage
> • Realtime: polling/offline manual → Supabase Realtime (websockets)
> • Infra: deploy simples → Docker Swarm em VPS própria
> • Modelo: gratuito → SaaS com planos Free / Pro / Team
> • PWA: opcional → obrigatório (offline first)

---

2. Modelo de Negócio SaaS

2.1 Planos e Preços

Recurso	Free	Pro	Team
Preço	R$ 0 / mês	R$ 19 / mês	R$ 49 / mês
Notas	Até 100	Ilimitadas	Ilimitadas
Armazenamento (anexos)	50 MB	10 GB	50 GB
Histórico de versões	7 dias	Ilimitado (50 versões)	Ilimitado (50 versões)
Busca full-text	√	√	√
Tags	Até 10	Ilimitadas	Ilimitadas
Sync em tempo real	√	√	√
Uso offline (PWA)	√	√	√
Compartilhamento de notas	✗	✗	√
Membros por workspace	1	1	Até 20
Exportar (ZIP + Markdown)	√	√	√
Suporte	Comunidade	Email (48h)	Email prioritário (12h)

2.2 Gestão de Cotas

O backend verifica cotas antes de cada operação crítica. Ao atingir o limite, a API retorna HTTP 402 com código QUOTA_EXCEEDED e uma mensagem sugerindo upgrade.

- Contagem de notas: campo notes_count na tabela user_plans, incrementado/decrementado via trigger no PostgreSQL
- Armazenamento: campo storage_used_bytes, atualizado a cada upload/delete no Supabase Storage
- O frontend exibe barra de progresso de uso na página de configurações
- Email automático ao atingir 80% e 100% do limite (via Resend ou SendGrid)

2.3 Pagamentos

- Integração com Stripe (recomendado) ou Pagar.me (para foco no mercado brasileiro)
- Webhook do Stripe atualiza o plano do usuário na tabela user_plans em tempo real
- Trial de 14 dias do plano Pro para novos cadastros
- Downgrade automático para Free ao cancelar assinatura (sem perda de dados, apenas acesso restrito)

3. Stack Tecnológica

3.1 Backend

Componente	Tecnologia	Justificativa
Runtime	Node.js 20 LTS	Cross-platform, ecosystem maduro
Framework	Fastify 4	Alta performance, validação nativa com JSON Schema
ORM / Client	Supabase JS SDK + Prisma	SDK para Auth/Storage/Realtime; Prisma para queries complexas
Banco de dados	Supabase (PostgreSQL 15)	Gerenciado, RLS nativo, full-text search, Realtime embutido
Autenticação	Supabase Auth + Google OAuth	OAuth 2.0 gerenciado, sem senha para o usuário
Armazenamento	Supabase Storage	S3-compatible, CDN, políticas por bucket, substitui MinIO
Realtime	Supabase Realtime	Websockets gerenciados, sync entre abas e dispositivos
Pagamentos	Stripe	Webhooks, assinaturas recorrentes, portal do cliente
Email transacional	Resend	API simples, templates React Email, alto deliverability
Validação	Zod	Schema compartilhado entre API e frontend
Testes	Vitest + Supertest	Rápido, compatível com ESM

3.2 Frontend

Componente	Tecnologia	Justificativa
Framework	React 18	Componentes reutilizáveis, ecossistema amplo
Build tool	Vite + vite-plugin-pwa	HMR instantâneo, Service Worker automático
Estado global	Zustand	Simples, sem boilerplate excessivo
Roteamento	React Router v6	Padrão da indústria
Supabase client	@supabase/supabase-js	Auth, Storage, Realtime direto no frontend

Componente	Tecnologia	Justificativa
Estilização	Tailwind CSS	Utility-first, consistência visual
Editor	CodeMirror 6	Markdown nativo, leve, extensível
Offline storage	IndexedDB (idb)	Notas offline, fila de sync pendente
PWA	vite-plugin-pwa + Workbox	Service Worker, cache offline, instalável

3.3 Infraestrutura (Docker Swarm)

Componente	Tecnologia	Justificativa
Orquestração	Docker Swarm	Mais simples que Kubernetes para VPS única; suporte nativo a rolling updates
VPS	Hetzner / DigitalOcean	Custo-benefício, datacenter na Europa/EUA
Reverse proxy	Traefik v3	Auto-discovery de serviços Swarm, TLS automático via Let's Encrypt
Registry	Docker Hub ou GHCR	Armazenar imagens para deploy
Secrets	Docker Secrets	Injeção segura de credenciais nos containers
CI/CD	GitHub Actions	Build, test e deploy automático ao fazer push na main
Monitoramento	Uptime Kuma	Health checks, alertas por Telegram/email
Logs	Loki + Grafana	Centralização de logs de todos os serviços Swarm

---

4. Supabase — Arquitetura Detalhada

4.1 Autenticação com Google OAuth

Todo o fluxo de autenticação é delegado ao Supabase Auth. O frontend nunca manipula senhas.

1. Usuário clica em 'Entrar com Google'
2. Frontend chama supabase.auth.signInWithOAuth({ provider: 'google' })
3. Supabase redireciona para o Google OAuth consent screen
4. Google retorna code para o callback URL do Supabase
5. Supabase troca o code por tokens e cria/atualiza o registro na tabela auth.users
6. Frontend recebe a sessão via onAuthStateChange e armazena o access_token em memória

7. O access_token do Supabase é enviado no header Authorization de todas as chamadas à API Fastify
8. A API valida o token usando supabase.auth.getUser(token) a cada request

> **Configuração necessária no Supabase Dashboard**
> Authentication → Providers → Google → habilitar
> Adicionar Client ID e Client Secret do Google Cloud Console
> Redirect URL: https://[project].supabase.co/auth/v1/callback
> Site URL: https://app.cloudnotes.com.br
> Additional redirect URLs: http://localhost:5173 (desenvolvimento)

4.2 Row Level Security (RLS)

Todas as tabelas têm RLS habilitado. Mesmo que a API seja bypassada, um usuário nunca acessa dados de outro.

```
-- Política padrão para a tabela notes
CREATE POLICY "users_own_notes" ON notes
  FOR ALL USING (auth.uid() = user_id);
```

```
-- Política para note_versions
CREATE POLICY "users_own_versions" ON note_versions
  FOR ALL USING (
    note_id IN (SELECT id FROM notes WHERE user_id = auth.uid())
  );
```

4.3 Supabase Storage — Buckets

Bucket	Acesso	Conteúdo
note-attachments	Privado (signed URLs)	Imagens, PDFs e arquivos anexados às notas
avatars	Público	Fotos de perfil dos usuários
exports	Privado (signed URLs)	ZIPs de exportação gerados sob demanda

Política do bucket note-attachments: apenas o dono da nota pode fazer upload/download. URLs pré-assinadas expiram em 1 hora.

```
-- Storage policy
CREATE POLICY "owner_attachments" ON storage.objects
  FOR ALL USING (
    bucket_id = 'note-attachments'
    AND auth.uid()::text = (storage.foldername(name))[1]
  );
```

4.4 Realtime — Sincronização entre Abas e Dispositivos

O Supabase Realtime usa websockets para propagar mudanças no banco em tempo real. Isso elimina a necessidade de polling e resolve conflitos de edição entre abas.

- O frontend subscreve ao canal notes:{user_id} ao fazer login
- Qualquer INSERT, UPDATE ou DELETE na tabela notes dispara um evento para todos os clientes conectados do mesmo usuário
- O frontend atualiza o store Zustand ao receber o evento, sem precisar refazer a query
- Se o usuário estiver editando a nota que foi atualizada em outro dispositivo, exibe banner: 'Esta nota foi atualizada em outro dispositivo. Deseja recarregar?'

```
// Exemplo de subscrição no frontend
supabase.channel('notes')
  .on('postgres_changes', {
    event: '*', schema: 'public', table: 'notes',
    filter: `user_id=eq.${userId}`
  }, (payload) => handleRealtimeChange(payload))
  .subscribe()
```

---

5. Modelo de Dados (Supabase / PostgreSQL)

> **Nota sobre auth.users**
> A tabela de usuários principal é gerenciada pelo Supabase Auth (schema auth.users).
> Criamos uma tabela profiles no schema public com FK para auth.users,
> populada automaticamente via trigger ao criar conta.

5.1 Tabela: profiles

Coluna	Tipo	Restrição	Descrição
id	UUID	PK, FK → auth.users	Mesmo ID do Supabase Auth
username	VARCHAR(50)	UNIQUE	Nome de exibição (auto-gerado do Google)
avatar_url	TEXT	nullable	URL do avatar (Google ou Supabase Storage)
created_at	TIMESTAMPTZ	default NOW()	Data de criação
updated_at	TIMESTAMPTZ	auto-update	Última atualização

5.2 Tabela: user_plans

Coluna	Tipo	Restrição	Descrição
user_id	UUID	PK, FK → auth.users	Usuário
plan	ENUM	free|pro|team	Plano ativo
stripe_customer_id	TEXT	nullable	ID do cliente no Stripe
stripe_subscription_id	TEXT	nullable	ID da assinatura no Stripe
trial_ends_at	TIMESTAMPTZ	nullable	Fim do período trial (14 dias)
notes_count	INTEGER	default 0	Contador de notas ativas
storage_used_bytes	BIGINT	default 0	Storage usado em bytes
current_period_end	TIMESTAMPTZ	nullable	Próxima cobrança

5.3 Tabela: notes

Coluna	Tipo	Restrição	Descrição
id	UUID	PK, default gen_random_uuid()	Identificador único
user_id	UUID	FK → auth.users, NOT NULL	Dono da nota (RLS)
title	VARCHAR(500)	NOT NULL, default ''	Título extraído automaticamente
content	TEXT	NOT NULL, default ''	Conteúdo em Markdown
content_tsv	TSVECTOR	generated always	Índice full-text search automático
is_pinned	BOOLEAN	default false	Nota fixada no topo
is_archived	BOOLEAN	default false	Nota arquivada
is_deleted	BOOLEAN	default false	Soft delete
deleted_at	TIMESTAMPTZ	nullable	Data do soft delete
word_count	INTEGER	generated always	Calculado automaticamente
created_at	TIMESTAMPTZ	default NOW()	Data de criação
updated_at	TIMESTAMPTZ	auto-update	Última modificação

Coluna	Tipo	Restrição	Descrição
		via trigger	

5.4 Tabelas: tags, note_tags, note_versions, attachments

Mantidas conforme a v1.0, com a seguinte alteração: todas as FKs de user_id apontam para auth.users ao invés de uma tabela users local.

---

6. Especificação da API REST

Base URL: https://api.cloudnotes.com.br/v1

Autenticação: todas as rotas (exceto webhooks) requerem o header:

```
Authorization: Bearer <supabase_access_token>
```

A API valida o token usando o Supabase Admin SDK. Não há JWT próprio — o token é o emitido pelo Supabase Auth após o OAuth.

6.1 Autenticação

> **Simplificação em relação à v1.0**
> As rotas /auth/register, /auth/login, /auth/refresh e /auth/logout foram removidas.
> Todo o fluxo de auth é gerenciado pelo Supabase Auth no frontend.
> A API mantém apenas /auth/me para retornar o perfil e plano do usuário autenticado.

Método	Endpoint	Auth	Descrição
GET	/auth/me	Supabase JWT	Retorna perfil + plano + uso de cotas

6.2 Notas

Método	Endpoint	Auth	Descrição
GET	/notes	JWT	Listar notas (paginado, com filtros)
POST	/notes	JWT	Criar nota (verifica cota do plano)
GET	/notes/:id	JWT	Buscar nota por ID
PUT	/notes/:id	JWT	Atualizar nota
DELETE	/notes/:id	JWT	Soft delete
DELETE	/notes/:id/permanent	JWT	Exclusão permanente
POST	/notes/:id/restore	JWT	Restaurar da lixeira
POST	/notes/:id/pin	JWT	Fixar/desafixar

Método	Endpoint	Auth	Descrição
POST	/notes/:id/archive	JWT	Arquivar/desarquivar
GET	/notes/:id/versions	JWT	Histórico de versões (limitado pelo plano)
POST	/notes/:id/duplicate	JWT	Duplicar nota

6.3 Tags

Método	Endpoint	Auth	Descrição
GET	/tags	JWT	Listar tags do usuário
POST	/tags	JWT	Criar tag (Free: máx 10)
PUT	/tags/:id	JWT	Renomear ou trocar cor
DELETE	/tags/:id	JWT	Remover tag

6.4 Busca

Método	Endpoint	Auth	Descrição
GET	/search	JWT	Full-text search via PostgreSQL TSVECTOR

```
GET /search?q=react+hooks&limit=10&highlight=true
```

6.5 Anexos

Método	Endpoint	Auth	Descrição
POST	/notes/:id/attachments	JWT	Upload para Supabase Storage (verifica cota)
GET	/notes/:id/attachments	JWT	Listar anexos da nota
DELETE	/attachments/:id	JWT	Remover do Storage e do banco
GET	/attachments/:id/url	JWT	Gerar signed URL (1h de validade)

6.6 Planos e Assinaturas

Método	Endpoint	Auth	Descrição
GET	/billing/plans	Público	Listar planos e preços
POST	/billing/checkout	JWT	Criar sessão de checkout no Stripe
POST	/billing/portal	JWT	Abrir portal de gerenciamento do Stripe
POST	/billing/webhook	Stripe signature	Receber eventos do Stripe (sem auth JWT)

Método	Endpoint	Auth	Descrição
GET	/billing/usage	JWT	Retornar uso atual (notas, storage)

6.7 Usuário

Método	Endpoint	Auth	Descrição
PUT	/user/profile	JWT	Atualizar username e avatar
DELETE	/user/account	JWT	Excluir conta e todos os dados
GET	/user/export	JWT	Exportar notas em ZIP (JSON + MD)

---

7. PWA — Offline First

7.1 Estratégia de Cache (Workbox)

Recurso	Estratégia	Detalhe
Assets estáticos (JS, CSS, fonts)	Cache First	Hash no nome do arquivo; cache permanente
HTML principal (index.html)	Network First	Fallback para cache se offline
Imagens de avatares	Stale While Revalidate	Serve do cache enquanto atualiza em background
Chamadas à API (/notes)	Network First + IndexedDB	Resposta da API salva no IDB; serve IDB se offline
Supabase Realtime	Skip Waiting	Conexão websocket; reconecta ao voltar online

7.2 Sincronização Offline

9. Usuário fica offline → Service Worker detecta via navigator.onLine
10. Edições são salvas em IndexedDB com flag { pendingSync: true, localVersion: n }
11. Banner 'Sem conexão — mudanças serão sincronizadas' aparece na UI
12. Conexão restaurada → Background Sync API dispara o evento sync
13. Service Worker percorre fila pendingSync e envia as edições em ordem cronológica
14. Conflito detectado (updatedAt do servidor > updatedAt local): versão do servidor prevalece, versão local salva como rascunho no histórico
15. Banner 'Tudo sincronizado √' aparece por 3 segundos

7.3 Instalação (Manifest)

- Ícones: 192x192 e 512x512 (PNG)

- display: standalone — sem barra do browser
- orientation: any — suporte a rotação no mobile
- theme_color: #211e1a (tema escuro), background_color: #f7f6f3 (tema claro)
- screenshots: obrigatório para prompt de instalação no Android Chrome
- Testado e instalável: Android Chrome, iOS Safari, Windows Edge, macOS Safari, Linux Chrome

---

8. Docker Swarm — Infraestrutura

8.1 Topologia

VPS única com Docker Swarm no modo single-node (manager = worker). Permite escalar para múltiplos nós no futuro sem mudança de arquitetura.

Serviços no stack Swarm

traefik → Reverse proxy + TLS automático (Let's Encrypt)
api → Backend Fastify (2 réplicas)
web → Frontend React/PWA servido pelo Nginx (2 réplicas)
uptime-kuma → Health checks e alertas
loki → Coleta de logs
grafana → Dashboard de logs e métricas

8.2 Arquivo docker-stack.yml (estrutura)

```
version: '3.9'
services:
  traefik:
    image: traefik:v3
    ports: ['80:80', '443:443']
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - traefik-certs:/certs
    deploy:
      placement:
        constraints: [node.role == manager]

  api:
    image: ghcr.io/seu-usuario/cloudnotes-api:latest
    secrets: [supabase_url, supabase_service_key, stripe_secret]
    deploy:
      replicas: 2
      update_config: { parallelism: 1, delay: 10s }
      restart_policy: { condition: on-failure }
    labels:
```

```
      - traefik.http.routers.api.rule=Host(`api.cloudnotes.com.br`)
      - traefik.http.routers.api.tls.certresolver=letsencrypt

  web:
    image: ghcr.io/seu-usuario/cloudnotes-web:latest
    deploy:
      replicas: 2
    labels:
      - traefik.http.routers.web.rule=Host(`app.cloudnotes.com.br`)
      - traefik.http.routers.web.tls.certresolver=letsencrypt
```

8.3 Secrets (Docker Secrets)

Secret	Conteúdo
supabase_url	URL do projeto Supabase (ex: https://xyz.supabase.co)
supabase_service_key	Service role key do Supabase (nunca exposta ao frontend)
supabase_anon_key	Anon key (pública, mas gerenciada como secret por consistência)
stripe_secret_key	sk_live_... do Stripe
stripe_webhook_secret	whsec_... para validar webhooks do Stripe
resend_api_key	API key do Resend para emails transacionais

8.4 CI/CD com GitHub Actions

16. Push na branch main → trigger do workflow
17. Build da imagem Docker da API e do frontend
18. Push das imagens para GHCR (GitHub Container Registry)
19. SSH na VPS → docker stack deploy --with-registry-auth cloudnotes
20. Swarm faz rolling update: derruba 1 réplica, sobe nova, verifica health, repete
21. Notificação no Telegram: deploy concluído ou falhou

---

9. Segurança

- Autenticação 100% delegada ao Supabase Auth — sem gerenciar senhas ou tokens manualmente
- RLS no Supabase: isolamento de dados por usuário a nível de banco de dados
- API valida token Supabase em cada request com getUser() — nunca confia apenas no payload JWT
- Docker Secrets: nenhuma credencial em variáveis de ambiente ou em arquivos versionados
- Traefik com TLS automático (Let's Encrypt) — HTTPS obrigatório em produção
- Headers de segurança via Traefik middlewares: HSTS, X-Frame-Options, CSP

- Rate limiting: 20 req/s por IP via plugin Fastify-rate-limit
- Supabase Storage: signed URLs com expiração de 1h; buckets privados por padrão
- Stripe: validação de assinatura do webhook com stripe.webhooks.constructEvent()
- CORS: whitelist explícita com origens permitidas (app.cloudnotes.com.br)

---

10. Variáveis de Ambiente

10.1 API

```
NODE_ENV=production
PORT=3001
SUPABASE_URL=https://[project-id].supabase.co
SUPABASE_SERVICE_KEY=<service_role_key>   # via Docker Secret
SUPABASE_ANON_KEY=<anon_key>
STRIPE_SECRET_KEY=sk_live_...             # via Docker Secret
STRIPE_WEBHOOK_SECRET=whsec_...           # via Docker Secret
RESEND_API_KEY=re_...                     # via Docker Secret
CORS_ORIGINS=https://app.cloudnotes.com.br
PLAN_FREE_MAX_NOTES=100
PLAN_FREE_MAX_STORAGE_MB=50
```

10.2 Frontend

```
VITE_SUPABASE_URL=https://[project-id].supabase.co
VITE_SUPABASE_ANON_KEY=<anon_key>          # pública, segura no frontend
VITE_API_URL=https://api.cloudnotes.com.br/v1
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_...
VITE_APP_NAME=CloudNotes
```

---

11. Estrutura de Pastas do Projeto

11.1 Repositório (monorepo)

```
cloudnotes/
  api/                  → Backend Fastify
  web/                  → Frontend React PWA
  infra/                → Docker Swarm stack, Traefik config
  .github/workflows/    → CI/CD GitHub Actions
  supabase/             → Migrations SQL e seed
```

11.2 API (api/)

```
src/
```

```
  modules/
    auth/                → Middleware de validação do token Supabase
    notes/               → CRUD + verificação de cotas
    tags/                → CRUD de tags
    attachments/         → Upload/download via Supabase Storage
    billing/             → Stripe checkout, portal, webhooks
    users/               → Perfil, exportação, exclusão
    search/              → Full-text search via TSVECTOR
  plugins/               → Fastify plugins (supabase, stripe, resend)
  lib/                   → Helpers, constantes, erros, quotas
```

11.3 Frontend (web/)

```
src/
  pages/                 → LoginPage, RegisterPage (OAuth), NoteEditorPage,
SettingsPage, PricingPage
  components/
    editor/              → NoteEditor, Toolbar, VersionHistory
    layout/              → AppLayout, Sidebar, NoteList
    billing/             → PricingCard, UsageBar, UpgradeModal
    ui/                  → Button, Input, Modal, Toast
  store/                 → Zustand (auth, notes, tags, ui, billing)
  hooks/                 → useNotes, useSearch, useRealtime, useOfflineSync
  services/              → api.js (Axios), supabase.js (client)
  lib/                   → utils, markdown, quotas
```

---

12. Fluxos Principais

12.1 Primeiro Acesso (Google OAuth)

22. Usuário acessa app.cloudnotes.com.br → vê tela de login
23. Clica em 'Entrar com Google' → redirecionado para Google OAuth
24. Autoriza → Google redireciona para callback do Supabase
25. Supabase cria registro em auth.users + dispara trigger que cria profiles e user_plans (plano Free)
26. Frontend recebe sessão via onAuthStateChange → redireciona para /
27. Tela de boas-vindas exibe tutorial rápido e cria primeira nota de exemplo

12.2 Upgrade Free → Pro

28. Usuário tenta criar a 101ª nota → API retorna 402 QUOTA_EXCEEDED
29. Frontend exibe modal de upgrade com comparativo de planos
30. Usuário clica em 'Assinar Pro' → frontend chama POST /billing/checkout
31. API cria sessão de checkout no Stripe e retorna a URL
32. Usuário é redirecionado para o Stripe Checkout (página do Stripe)

33. Pagamento aprovado → Stripe envia webhook customer.subscription.created para POST /billing/webhook
34. API atualiza user_plans.plan = 'pro' no banco
35. Usuário é redirecionado de volta para o app → pode criar notas ilimitadas

12.3 Sincronização Realtime

36. Usuário abre app na aba A e na aba B
37. Na aba A, edita e salva a nota X (PUT /notes/:id)
38. Supabase Realtime detecta o UPDATE na tabela notes
39. Dispara evento para todos os canais subscrito no user_id
40. Aba B recebe o evento → atualiza a nota X no store Zustand automaticamente
41. Se aba B também estava editando X → exibe banner de conflito

---

13. Critérios de Aceite (MVP)

- Login com Google OAuth funciona em < 5 segundos
- Novo usuário recebe plano Free automaticamente ao criar conta
- Notas são salvas automaticamente (auto-save com debounce de 1.5s)
- Full-text search retorna resultados em < 200ms
- Mudanças propagadas via Realtime para outra aba em < 1 segundo
- PWA instalável em Android, iOS, Windows e macOS
- App funciona offline e sincroniza ao reconectar
- Limite de 100 notas do plano Free é respeitado — API retorna 402 ao exceder
- Checkout do Stripe completa e plano é atualizado em < 30 segundos
- Deploy via docker stack deploy sem downtime (rolling update)
- TLS automático via Traefik + Let's Encrypt em produção
- RLS impede acesso a dados de outro usuário mesmo com token válido de terceiro

---

14. Fora do Escopo (v1.0 MVP)

- Colaboração em tempo real (edição simultânea por múltiplos usuários)
- App nativo iOS/Android (React Native) — a PWA cobre o MVP
- Login com email/senha ou GitHub OAuth
- OCR em imagens
- Notificações push
- Importação direta do Simplenote ou Evernote
- Multi-nó Docker Swarm (cluster com manager + workers separados)
- Editor WYSIWYG completo — foco em Markdown puro

— Fim do documento —